



AGENT INTELLIGENT D'ASSISTANCE AUX REUNIONS, AU PILOTAGE DE LA PERFORMANCE ET A LA GESTION DES EMAILS

Rapport de stage de fin d'année



PROFESSEUR : HAMID EL GHAZI
ETUDIANTS : SANAE SOUJAA & NIZAR ES-SAMY

Résumé

Ce projet s'inscrit dans le cadre de la transformation digitale des processus de pilotage et d'assistance au dirigeant. Il consiste à concevoir et à développer un **agent intelligent multi-tâches** capable d'assister un dirigeant d'entreprise dans la gestion quotidienne de ses réunions, le suivi des décisions et des actions, l'analyse de la performance commerciale et financière via le système d'information, ainsi que la gestion des emails professionnels et des documents Google Drive.

L'agent, développé autour d'un socle **LangGraph + LangChain**, s'appuie sur un modèle de langage **DeepSeek** et sur un ensemble d'outils (tools) connectés aux services Google (Gmail, Drive, Calendar), à une base vectorielle **ChromaDB** pour la mémoire de long terme, à un moteur de transcription audio **AssemblyAI**, et à un connecteur ERP simulant **SAP Business One**. La partie frontale est une application mobile/web développée en **React Native (Expo)**.

La solution couvre l'ensemble du périmètre fonctionnel du cahier des charges : préparation de réunions, génération automatique de procès-verbaux (PV), suivi des actions et des deadlines, analyse ERP (chiffre d'affaires, marges, retards, écarts objectifs/résultats), proposition de réponses aux emails avec validation humaine, transcription multilingue et challenge proactif des décisions.

Mots-clés : Agent intelligent, LLM, LangGraph, RAG, ChromaDB, ERP, Gmail, Google Drive, Google Calendar, React Native, FastAPI, AssemblyAI.

Abstract

This project is part of the digital transformation of executive assistance and performance management processes. It aims to design and develop a **multi-task intelligent agent** able to assist a business leader in the daily management of meetings, decision and action tracking, commercial and financial performance analysis through the information system (SAP Business One), as well as professional email and Google Drive document management.

The agent, built on a **LangGraph + LangChain** stack, relies on a **DeepSeek** Large Language Model and a set of tools connected to Google services (Gmail, Drive, Calendar), a **ChromaDB** vector database for long-term memory, an **AssemblyAI** audio transcription engine, and an ERP connector. The front-end is a mobile/web application built with **React Native (Expo)**.

The solution covers the entire functional scope of the specification: meeting preparation, automatic generation of meeting minutes, action and deadline tracking, ERP analytics (revenue, margins, delays, target/result gaps), semi-automatic email replies with human validation, multilingual transcription and proactive challenge of decisions.

Keywords: Intelligent agent, LLM, LangGraph, RAG, ChromaDB, ERP, Gmail, Google Drive, Google Calendar, React Native, FastAPI, AssemblyAI.

Table des matières

Résumé	2
Abstract	3
Introduction	5
Chapitre 1 — Analyse de l'existant et spécification du besoin.....	6
1.1 Étude de l'existant	6
1.2 Spécification du besoin.....	7
Chapitre 2 — Présentation de la solution	10
2.1 Choix des outils.....	10
2.2 Architecture logicielle	11
Chapitre 3 — Présentation des agents IA	14
3.1 Agent de gestion des emails et documents Drive	14
3.2 Agent de la situation ERP	17
3.3 Agent de gestion des réunions	19
3.4 Agent de gestion du calendrier.....	23
3.5 Fonctionnalité de transcription multi-langues.....	25
Conclusion.....	28
Apports du projet	29
Limites et perspectives	29

Introduction

La digitalisation des processus métiers est devenue un levier stratégique incontournable pour les organisations souhaitant gagner en réactivité, en traçabilité et en qualité de décision. Parmi les fonctions les plus critiques figure l'**assistance au dirigeant**, qui doit simultanément :

- préparer et animer les réunions,
- suivre les décisions et les actions,
- analyser la performance commerciale et financière,
- gérer une volumétrie croissante d'emails,
- et s'assurer que les engagements pris sont tenus.

Or, dans la majorité des PME et ETI, ces activités sont encore réalisées manuellement, avec des outils dispersés (Outlook, Excel, WhatsApp, ERP, Google Drive), ce qui entraîne des pertes de temps, un manque de traçabilité, une faible discipline d'exécution et une incapacité à relier les décisions aux résultats réels.

C'est dans ce contexte que s'inscrit notre projet : **concevoir un agent intelligent unifié**, capable d'orchestrer l'ensemble de ces activités autour d'un noyau d'intelligence artificielle générative, connecté au système d'information de l'entreprise (SAP Business One) et aux services collaboratifs (Gmail, Drive, Calendar).

L'objectif est triple :

1. **Gagner du temps** en automatisant les tâches répétitives (résumés d'emails, proposition de réponses, génération PV, préparation de réunions, relances).
2. **Améliorer la discipline d'exécution** en assurant le suivi systématique des décisions et en alertant sur les retards.
3. **Piloter proactivement** en reliant décisions, actions et impacts ERP, et en proposant des recommandations correctives.

Le présent rapport est structuré en trois chapitres :

- Le **premier chapitre** dresse l'analyse de l'existant et formalise le besoin.
- Le **deuxième chapitre** présente la solution retenue, les outils choisis et l'architecture logicielle.
- Le **troisième chapitre** détaille, agent par agent, le fonctionnement de la solution.

Chapitre 1 — Analyse de l'existant et spécification du besoin

1.1 Étude de l'existant

1.1.1 Système actuel

Dans la plupart des organisations ciblées par ce projet, la gestion des réunions, des emails et du pilotage de la performance repose aujourd'hui sur un ensemble d'outils hétérogènes et faiblement intégrés :

Activité	Outil utilisé actuellement	Format
Emails	Gmail / Outlook	Boîte de réception classique
Documents	Google Drive / OneDrive	Dossiers partagés
Réunions	Google Meet / Teams / présentiel	Notes manuscrites ou Word
Procès-verbaux	Word / Google Docs	Rédaction manuelle
Suivi des actions	Excel / Trello / rien	Tableau statique
Agenda	Google Calendar	Événements manuels
Pilotage ERP	SAP Business One, Odoo,...	Consultation ponctuelle
Transcription audio	Outils isolés (Otter, Whisper...)	Hors écosystème

Le dirigeant jongle donc en permanence entre plusieurs applications, sans continuité entre les décisions prises en réunion, les actions assignées, les emails échangés et les résultats visibles dans l'ERP.

L'observation de terrain met en évidence plusieurs scénarios typiques :

- **Scénario 1** — Après une réunion commerciale, un PV est rédigé manuellement dans Word, envoyé par email, puis... oublié. Les actions ne sont ni suivies ni rappelées.

- **Scénario 2** — Un client envoie une demande de vérification de paiement. L’assistant cherche dans Gmail, puis dans SAP, puis dans les PDF archivés sur Drive. La réponse prend plusieurs heures.
- **Scénario 3** — Le dirigeant veut savoir « où en est le business ce mois ? ». Il doit ouvrir SAP, exporter des données, croiser avec Excel, et reconstituer une vue globale.
- **Scénario 4** — Un enregistrement audio de réunion est transcrit par un outil externe, mais la transcription n’est ni indexée, ni résumée, ni reliée aux décisions, ni exploitable pour générer un PV structuré.

1.1.2 Limites du système actuel

L’analyse du système actuel fait ressortir les limites suivantes :

1. **Dispersion de l’information** — Les données sont éclatées entre Gmail, Drive, Calendar, SAP et fichiers locaux, sans couche d’unification.
2. **Absence de mémoire organisationnelle** — Les décisions et actions passées ne sont ni indexées ni retrouvables de manière sémantique.
3. **Faible automatisation** — Les résumés d’emails, PV, relances, analyses ERP sont produits manuellement.
4. **Aucune liaison décision → impact** — Il est impossible de savoir si une décision prise en réunion a réellement produit un effet mesurable dans l’ERP.
5. **Manque de proactivité** — Aucun mécanisme d’alerte automatique en cas d’action en retard, de deadline proche ou d’écart objectif/résultat.
6. **Transcription non intégrée** — Les enregistrements audio de réunions ne sont pas exploités de manière structurée et multilingue.
7. **Absence de validation humaine contrôlée** — Aucun garde-fou n’encadre l’envoi automatique d’emails sensibles.
8. **Manque de traçabilité** — Aucun journal ne permet de savoir quelles actions l’agent a réalisées, à quel moment et sur quelles données.

Ces limites justifient pleinement la conception d’un agent intelligent unifié, doté d’une mémoire vectorielle, connecté aux services métiers et capable de raisonner sur plusieurs sources pour produire des livrables à forte valeur ajoutée.

1.2 Spécification du besoin

Le cahier des charges définit un périmètre fonctionnel structuré autour de **quatre axes majeurs** :

Axe 1 — Gestion intelligente des réunions

L'agent doit :

- Gérer plusieurs **types de réunions** : commerciales (responsables de zones, grands comptes), marketing, financières, stratégiques (6–7 personnes), hebdomadaires/bi-hebdomadaires, ponctuelles.
- **Avant la réunion** : proposer un ordre du jour structuré, rappeler les décisions et actions de la réunion précédente, identifier les points critiques, suggérer des questions de pilotage, proposer un animateur.
- **Pendant/juste après** : extraire automatiquement participants, points abordés, décisions, actions à partir d'un audio, de notes ou d'un compte rendu brut.
- **Après la réunion** : générer un **procès-verbal** structuré et professionnel (informations générales, points abordés, décisions, plan d'action détaillé avec responsable/échéance/priorité/KPI).
- **Diffusion et suivi** : envoyer le PV par email, archiver par type/service, relancer 2–3 jours avant la prochaine réunion, alerter en cas de retard.

Axe 2 — Plan d'action intelligent et rôle de « challenge »

L'agent ne doit pas être passif. Il doit :

- Proposer un **plan d'action complémentaire** aux décisions.
- Identifier **incohérences, risques, actions manquantes**.
- **Challenger** les décisions : poser des questions, proposer des alternatives, suggérer des améliorations opérationnelles.

Axe 3 — Connexion au système d'information (SAP Business One)

- Se connecter à SAP B1 (via API ou exports sécurisés).
- Accéder aux données clés : **chiffre d'affaires, réalisations commerciales, marges, retards/écarts**.
- **Relier les décisions de réunion à leurs impacts réels**.
- Identifier les **écarts entre objectifs et résultats**.

- Alerter en cas de non-atteinte des objectifs et proposer des plans d'action correctifs.

Axe 4 — Gestion intelligente des emails et documents

- Lire et analyser les emails entrants.
- **Proposer des réponses** semi-automatiques avec **validation humaine obligatoire** avant envoi.
- Adapter le ton au contexte (commercial, administratif, stratégique).
- Identifier les emails nécessitant une action.
- Classer automatiquement les emails (par client, projet, type).
- Historiser les échanges, offres envoyées, relances effectuées.
- **Suivre l'état d'avancement des dossiers commerciaux** : demande → offre envoyée → relance → négociation → décision finale.

Contraintes transverses

- **Sécurité et confidentialité** : accès sécurisé aux données (OAuth2), droits différenciés.
 - **Gouvernance** : journal des actions de l'agent, **validation humaine** avant envoi d'emails sensibles.
 - **Multilinguisme** : transcription réussie même quand l'audio est multilingue.
 - **Évolutivité** : architecture modulaire permettant le remplacement des composants (ex. données SAP mock → SAP réel).
-

Chapitre 2 — Présentation de la solution

2.1 Choix des outils

La solution retenue repose sur un ensemble de technologies modernes, open-source ou facilement accessibles, choisies pour leur robustesse, leur modularité et leur compatibilité avec les contraintes du projet.

2.1.1 Côté backend (Agent IA)

Composant	Technologie	Justification
Framework agent	LangGraph + LangChain	Orchestration d’agents avec tools, gestion d’état, compatibilité LLM
Modèle de langage	DeepSeek (deepseek-v4-flash) via API OpenAI-compatible	Excellent rapport coût/performance, support multilingue, API compatible OpenAI
Base vectorielle	ChromaDB	Base locale, persistante, embeddings via Sentence Transformers
Embeddings	all-MiniLM-L6-v2	Rapide, léger, performant en multilingue
API REST	FastAPI + Uvicorn	Framework Python moderne, asynchrone, doc OpenAPI automatique
Scheduler	schedule + threading	Traitement horaire automatique des emails
Transcription	AssemblyAI	Support multilingue natif, gestion des accents, simple
Google APIs	google-api-python-client, google-auth-oauthlib	Accès sécurisé à Gmail, Drive, Calendar via OAuth2
ERP	SAP Business One (mock + connecteur futur)	Conformité au cahier des charges

2.1.2 Côté frontend

Composant	Technologie	Justification
Framework UI	React Native + Expo	Multiplateforme (web + mobile), développement rapide
Gestion d'état	React Hooks (useState, useRef)	Simplicité, suffisant pour la portée du projet
HTTP	Axios	Requêtes REST vers FastAPI
Rendu Markdown	react-native-markdown-display	Affichage riche des réponses de l'agent
Sélecteur de fichiers	expo-document-picker	Upload d'audio pour transcription

2.1.3 Outils de développement

- **Python 3.10+** pour le backend
- **Node.js 18+** pour le frontend
- **Git / GitHub** pour le versioning
- **Postman / Swagger** pour tester les endpoints
- **VS Code** comme IDE principal

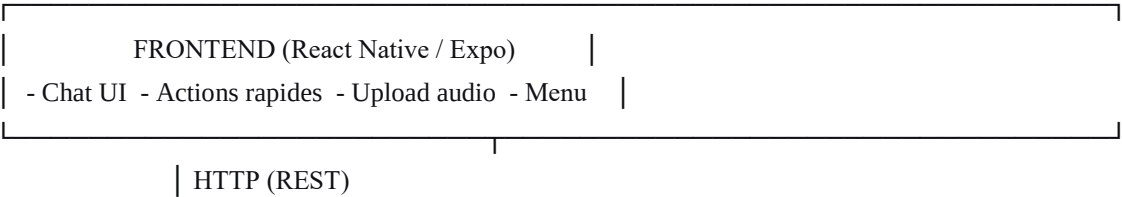
2.2 Architecture logicielle

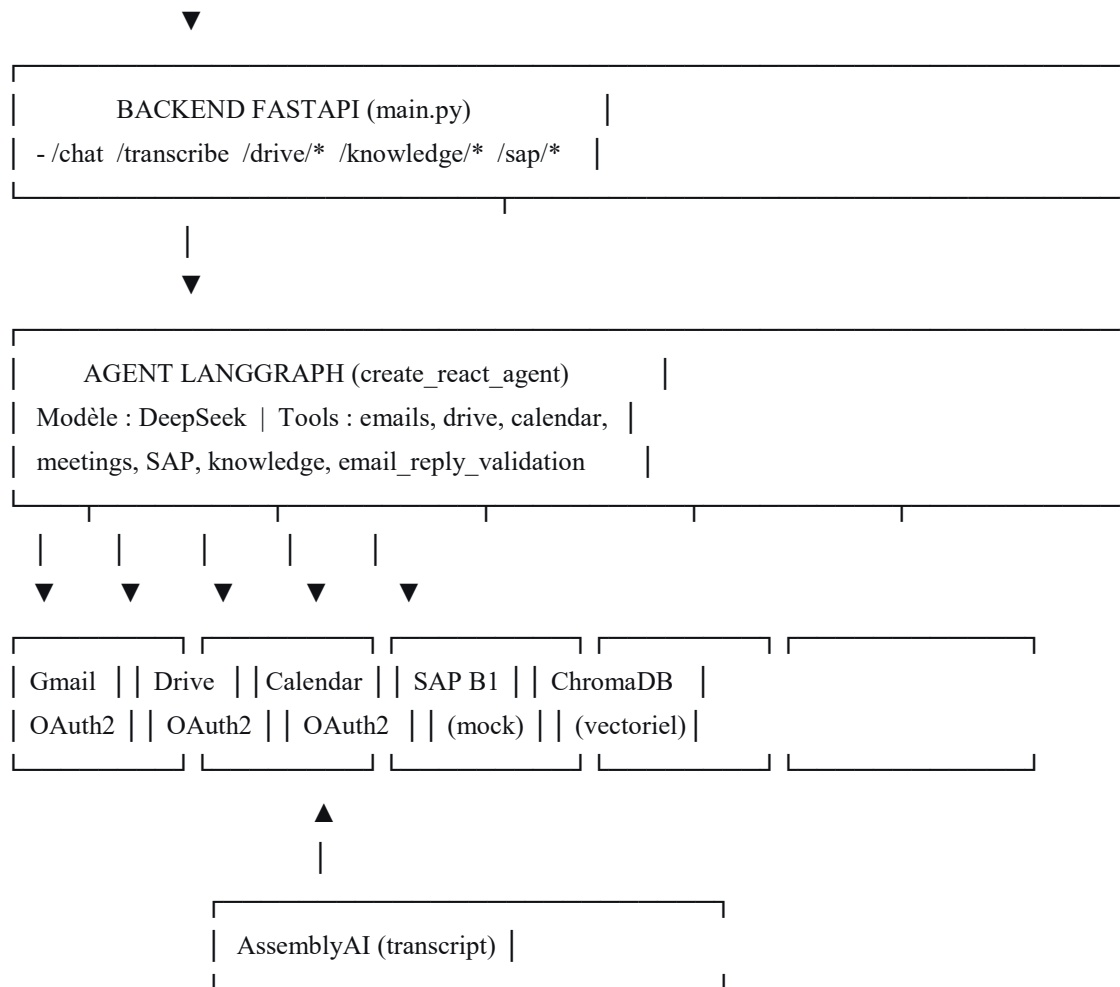
L'architecture retenue est **modulaire, en couches**, et repose sur une séparation claire entre :

- la **couche présentation** (frontend React Native),
- la **couche orchestration** (FastAPI + agent LangGraph),
- la **couche outils** (tools : emails, Drive, Calendar, réunions, SAP),
- la **couche données** (ChromaDB, fichiers JSON de suivi, Google APIs, SAP),
- la **couche services externes** (DeepSeek, AssemblyAI, Google OAuth2).

2.2.1 Vue d'ensemble

text





2.2.2 Structure des fichiers du projet

Fichier	Rôle
<code>main.py</code>	Point d'entrée FastAPI + création de l'agent + endpoints REST
<code>config.py</code>	Configuration commune (clés API, ChromaDB, MeetingManager)
<code>emails.py</code>	Authentification Gmail, résumé, indexation ChromaDB
<code>drive.py</code>	Synchronisation Drive → extraction → ChromaDB
<code>google_calendar.py</code>	Outils Calendar (lecture, création, modification, suppression)
<code>meetings.py</code>	Classe <code>MeetingManager</code> (préparation, PV, suivi)
<code>meeting_tools.py</code>	Tools LangChain exposés à l'agent

Fichier	Rôle
<code>email_reply_validation.py</code>	Préparation et validation humaine des réponses emails
<code>sap_mock_data.py</code>	Données SAP B1 simulées (clients, produits, factures, objectifs)
<code>sap_tools.py</code>	Tools SAP exposés à l'agent (CA, marges, retards, écarts)
<code>transcription.py</code>	Module de transcription AssemblyAI
<code>App.js</code>	Application React Native (chat UI + upload audio)
<code>app.json</code>	Configuration Expo
<code>pending_email_replies.json</code>	Fichier de suivi des réponses en attente de validation
<code>processed_emails.json</code>	Suivi des emails déjà traités
<code>processed_drive_files.json</code>	Suivi des fichiers Drive déjà indexés

2.2.3 Flux d'une requête utilisateur

1. L'utilisateur saisit un message dans l'application mobile.
 2. Le frontend envoie une requête `POST /chat` avec le message et l'historique.
 3. FastAPI transmet le tout à l'**agent LangGraph**.
 4. L'agent **raisonne** (ReAct) : il décide quel(s) tool(s) appeler.
 5. Le(s) tool(s) exécutent la logique métier
(ex. `get_sap_snapshot`, `search_knowledge`, `prepare_email_reply`).
 6. L'agent formule une réponse finale en langage naturel.
 7. FastAPI renvoie la réponse + la liste des tools appelés.
 8. Le frontend affiche la réponse (avec rendu Markdown) et les tools utilisés.
-

Chapitre 3 — Présentation des agents IA

L'agent principal est un **agent ReAct** (Reasoning + Acting) construit avec `create_react_agent` de LangGraph. Il dispose d'un **prompt système** très détaillé qui décrit l'ensemble des tools disponibles et les règles de routing.

Chaque grande famille fonctionnelle est implémentée comme un **ensemble de tools** que l'agent peut invoquer. Nous présentons ci-dessous, agent par agent, les fonctionnalités développées.

3.1 Agent de gestion des emails et documents Drive

3.1.1 Objectifs

- Récupérer les emails récents depuis Gmail (OAuth2).
- Les **résumer** avec DeepSeek.
- Les **indexer** dans ChromaDB pour recherche sémantique.
- **Rechercher** dans les emails passés.
- **Synchroniser** Google Drive (Docs, Sheets, Slides, PDF, DOCX, XLSX, TXT, audio).
- **Extraire et indexer** le contenu des documents Drive.
- **Préparer des réponses emails** avec validation humaine obligatoire.

3.1.2 Tools développés

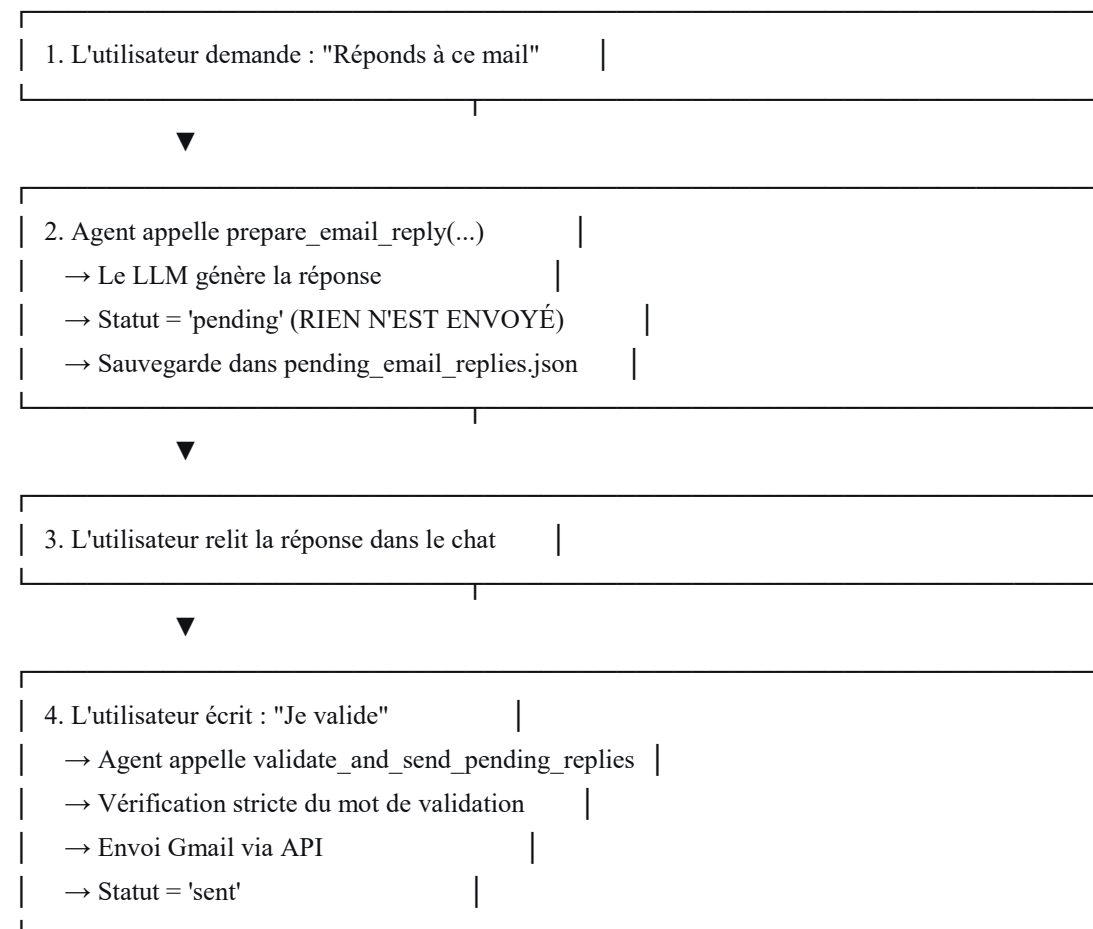
Tool	Rôle
<code>process_and_summarize_emails</code>	Récupère les nouveaux emails Gmail, les résume et les stocke dans ChromaDB
<code>search_emails</code>	Recherche sémantique dans les emails indexés
<code>summarize_email</code>	Résume un email donné
<code>list_drive_files</code>	Liste les fichiers accessibles dans Google Drive
<code>sync_drive_tool</code>	Synchronise Google Drive → extraction → ChromaDB
<code>summarize_drive_document</code>	Résume un document Drive

Tool	Rôle
<code>search_knowledge</code>	Recherche transverse (emails + Drive + transcriptions + réunions)
<code>prepare_email_reply</code>	Prépare une réponse email et la met en attente
<code>get_pending_email_replies</code>	Liste les réponses en attente
<code>validate_and_send_pending_replies</code>	Envoie une réponse uniquement après validation explicite
<code>cancel_pending_email_reply</code>	Annule une réponse en attente

3.1.3 Workflow de réponse email avec validation humaine

Ce workflow répond directement à l'exigence « **validation humaine possible avant envoi d'emails sensibles** » du cahier des charges.

text



Mots de validation acceptés (liste blanche stricte) : « je valide », « valide », « envoie », « envoyer », « tu peux envoyer », « envoie la réponse », « envoie le mail »...

Toute autre formulation entraîne un **refus d'envoi** et une demande de confirmation explicite. Ce mécanisme garantit qu'aucun email sensible n'est envoyé sans action délibérée de l'utilisateur.

3.1.4 Extraction de contenu Drive

Le module `drive.py` gère l'extraction pour plusieurs formats :

Format	Méthode d'extraction
PDF	<code>pypdf.PdfReader</code>
DOCX	<code>python-docx</code> (paragraphes + tableaux)
XLSX	<code>openpyxl</code> (toutes les feuilles)
TXT / MD / CSV / JSON / XML / HTML	Lecture directe avec fallback d'encodage
Google Docs	Export → DOCX puis extraction
Google Sheets	Export → CSV puis extraction
Google Slides	Export → TXT
Audio (MP3, M4A, WAV...)	Transcription AssemblyAI

Le contenu extrait est découpé en **chunks** (1200 caractères, overlap 200), vectorisé et stocké dans ChromaDB avec des métadonnées riches (`drive_file_id`, `filename`, `mime_type`, `category`, `modified_time`, `web_view_link`).

3.1.5 Suivi incrémental

Deux fichiers JSON assurent le suivi :

- `processed_emails.json` : liste des IDs d'emails déjà traités.
- `processed_drive_files.json` : dictionnaire `{file_id: {filename, modified_time, processed_at, chunks_count}}`.

Cela évite le retraitement inutile et permet la **réindexation automatique** lorsqu'un fichier Drive est modifié.

3.2 Agent de la situation ERP

3.2.1 Objectifs

Répondre aux exigences du **chapitre 4 du cahier des charges** : connexion au système d'information (SAP Business One), accès aux données clés (CA, réalisations, marges, retards), liaison décision → impact, identification des écarts objectifs/résultats, et génération de recommandations.

3.2.2 Architecture de la couche ERP

Dans l'attente du branchement réel à SAP B1 via le **Service Layer (OData)**, nous avons développé une couche de **données mock** (`sap_mock_data.py`) qui reproduit fidèlement la structure des tables SAP :

Table SAP	Équivalent mock
OCRD (Business Partners)	CUSTOMERS
OITM (Items)	PRODUCTS
OINV (Invoices)	INVOICES
OUDO / UDO (Targets)	TARGETS

Les fonctions `load_*` sont conçues pour être remplacées par des appels OData sans modifier les tools.

3.2.3 Tools SAP développés

Tool	Description
<code>get_sap_snapshot</code>	Vue globale (CA, marge, top clients, retards)
<code>get_sap_revenue</code>	CA sur une période avec regroupement (Region, SalesOwner, Category...)
<code>get_sap_sales_performance</code>	Réalisations vs objectifs par commercial
<code>get_sap_margins</code>	Analyse des marges par catégorie/produit/région
<code>get_sap_delays_and_gaps</code>	Retards de paiement/livraison et écarts
<code>link_decision_to_impact</code>	Relie une décision de réunion à son impact SAP réel
<code>analyze_objective_gap</code>	Identifie les écarts objectifs vs résultats

3.2.4 Périodes supportées

Les tools acceptent plusieurs formats de période : `current_month`, `last_30d`, `YYYY-MM`, `YYYY-MM-DD`, `YYYY-MM-DD`.

3.2.5 Exemple concret : « Où en est le business ce mois ? »

L'utilisateur pose la question dans le chat. L'agent appelle `get_sap_snapshot(period="current_month")`. Le tool agrège les factures du mois en cours et le résultat retourné est directement affiché dans l'interface sous forme de tableaux Markdown, comme le montre la capture suivante :



(Figure 1 — Interface chat affichant le snapshot SAP Business One : indicateurs clés et top clients.)

3.2.6 Liaison décision → impact

Le tool `link_decision_to_impact` détecte des mots-clés relative à une décision préalablement prise (ex. « grand compte », « Casablanca », « logiciel », « Karim ») et filtre les données SAP correspondantes. Il retourne :

- le CA concerné,
- la marge et le pourcentage,
- le nombre de documents,
- les retards détectés,
- une **interprétation automatique** (impact positif / modéré / faible).

C'est une fonctionnalité clé du cahier des charges (« Relier les décisions prises en réunion à leurs impacts réels »), qui distingue notre agent d'un simple chatbot.

3.3 Agent de gestion des réunions

3.3.1 Objectifs

Couvrir l'ensemble du périmètre du **chapitre 2 du cahier des charges** :

- Types de réunions : commerciales, marketing, financières, stratégiques, hebdomadaires, bi-hebdomadaires, ponctuelles.
- **Avant** : ordre du jour, rappel des décisions/actions précédentes, points critiques, animateur.
- **Pendant/après** : extraction participants, points, décisions, actions.
- **Après** : PV structuré et professionnel.
- **Diffusion et suivi** : envoi email, archivage, relances, alertes.

3.3.2 Modèle de données

Le module `meetings.py` définit des modèles **Pydantic** robustes :

- `MeetingType` (enum) : commercial, marketing, financier, strategique, hebdomadaire, bihebdomadaire, ponctuelle.
- `ActionPriority` : court_terme, moyen_terme, long_terme.
- `ActionStatus` : assignée, en_cours, bloquée, complétée, en_retard.
- `Action`, `Decision`, `Participant`, `MeetingPrepContext`, `MeetingMinutes`.

3.3.3 Tools réunions développés

Tool	Rôle
<code>prepare_meeting</code>	Prépare une réunion : agenda, contexte précédent, points critiques
<code>get_meeting_facilitator_suggestion</code>	Suggère un animateur
<code>analyze_meeting_transcript</code>	Analyse une transcription/notes pour extraire participants, points, décisions, a
<code>generate_meeting_minutes</code>	Génère un PV complet en HTML
<code>check_action_status</code>	Vérifie le statut d'une action
<code>update_action</code>	Met à jour le statut d'une action
<code>list_upcoming_deadlines</code>	Liste les deadlines des N prochains jours
<code>list_overdue_actions</code>	Liste les actions en retard

3.3.4 Génération d'ordre du jour adaptatif

Le MeetingManager propose des **agendas pré-structurés** par type de réunion. Exemple pour une réunion **commerciale** :

Ordre	Sujet	Durée
0	Suivi des actions précédentes (si applicable)	5 min
1	Synthèse des objectifs commerciaux	5 min
2	Progression par zone/client	20 min
3	Obstacles et ressources	15 min
4	Décisions et prochaines étapes	10 min
5	Questions ouvertes	10 min

L'agenda est enrichi dynamiquement par le rappel des actions non complétées issues de la réunion précédente, détectées via une requête ChromaDB.

3.3.5 Détection des points critiques

Le MeetingManager identifie automatiquement :

- les **actions en retard** (statut = `en_retard`),
- les **actions bloquées** (statut = `bloquée`),
- les **actions à haute priorité non complétées** (priorité = `court_terme`).

Il en déduit une **recommandation** du type : « ⚠ 3 actions critiques à résoudre.

Recommandation : traiter ces blocages en début de réunion. »

3.3.6 Génération de PV en HTML professionnel

Le PV généré contient :

1. **En-tête** : titre, date, type, animateur.
2. **Métadonnées** : participants, durée, prochaine réunion.

3. **Points abordés** (liste).
4. **Décisions prises** (tableau : décision, impact).
5. **Plan d'action** (tableau : action, responsable, échéance, priorité, statut, KPI).
6. **Pied de page** avec horodatage automatique.

Le HTML est stylé avec des couleurs distinctes par **statut** (complétée, en_cours, bloquée, assignée, en_retard) et par **priorité** (court/moyen/long terme), garantissant une lecture immédiate.

Le PV est ensuite **stocké dans ChromaDB** (collection `meetings_db`) pour permettre :

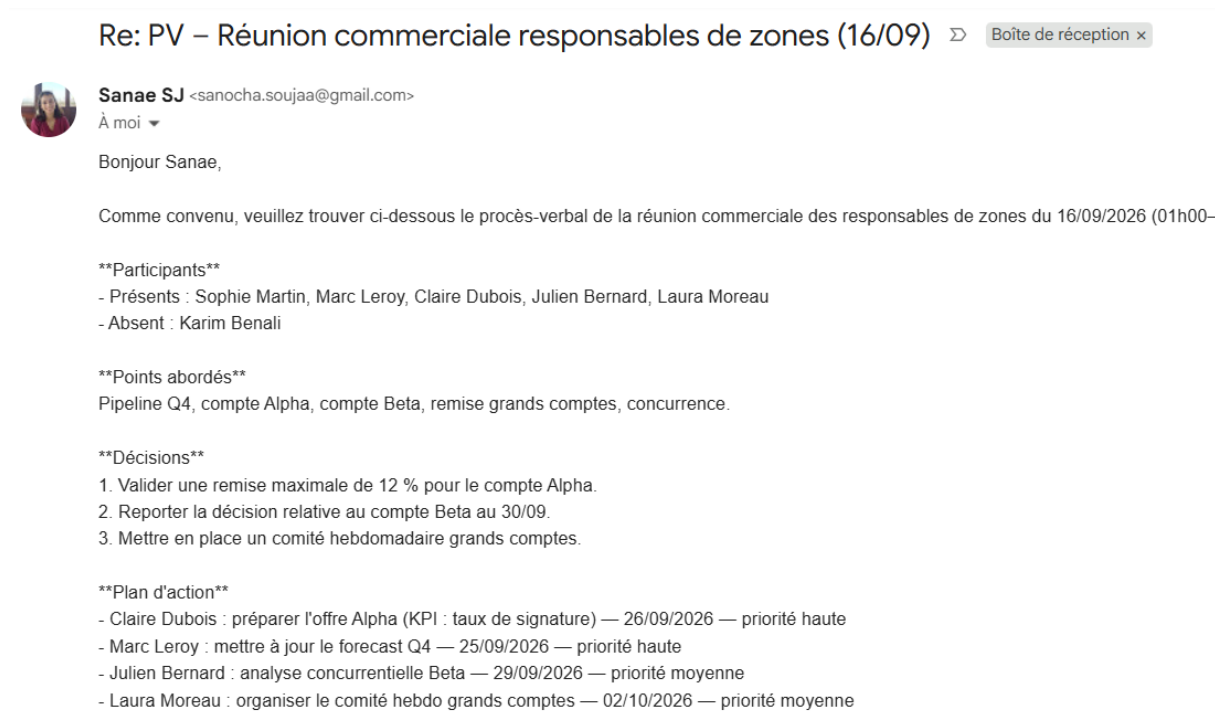
- la recherche ultérieure (« retrouve-moi la décision sur le compte Alpha »),
- la liaison avec les tools SAP (`link_decision_to_impact`),
- le rappel automatique lors de la prochaine réunion du même type.

3.3.7 Exemple concret de PV généré

Un PV de réunion commerciale du 16/09/2026 a été généré et envoyé par email. Il contenait :

- **Participants** : Sophie Martin, Marc Leroy, Claire Dubois, Julien Bernard, Laura Moreau (absent : Karim Benali).
- **Points abordés** : pipeline Q4, compte Alpha, compte Beta, remise grands comptes, concurrence.
- **Décisions** : remise max 12 % pour Alpha, report Beta au 30/09, comité hebdo grands comptes.
- **Plan d'action** : 4 actions assignées avec responsables, deadlines et priorités.
- **Prochaine réunion** : 07/10/2026, animée par Marc.

Le PV a été **envoyé par email** à la dirigeante après validation, illustrant le workflow complet **préparation → analyse → PV → diffusion → suivi**.



(Figure 2 — Capture d’écran du PV envoyé à l’animatrice de la réunion)

3.4 Agent de gestion du calendrier

3.4.1 Objectifs

Le module `google_calendar.py` fournit une intégration complète à **Google Calendar** via OAuth2, avec gestion du fuseau horaire **Africa/Casablanca**.

3.4.2 Tools développés

Tool	Rôle
<code>get_today_events</code>	Réunions du jour
<code>get_week_events</code>	Réunions des 7 prochains jours
<code>count_week_events</code>	Nombre de réunions de la semaine
<code>get_next_meeting</code>	Prochaine réunion

Tool	Rôle
<code>search_calendar</code>	Recherche par texte
<code>create_calendar_event</code>	Création d'une réunion
<code>update_calendar_event</code>	Modification
<code>delete_calendar_event</code>	Suppression

3.4.3 Normalisation des dates

Une fonction `normalize_calendar_datetime` accepte plusieurs formats ISO 8601 et applique automatiquement le fuseau **Africa/Casablanca** si aucune timezone n'est fournie. Elle supporte notamment :

- `2026-09-15T15:00:00` (heure locale Maroc),
- `2026-09-15T15:00:00+01:00`,
- `2026-09-15T14:00:00Z` (UTC).

Cette rigueur évite les décalages horaires, source classique de bugs dans les applications de calendrier.

3.4.4 Création d'événement avec Google Meet

Lors de la création d'un événement, le champ `conferenceData` est automatiquement renseigné pour générer un **lien Google Meet** :

```
python
"conferenceData": {
  "createRequest": {
    "requestId": f"{summary}-{datetime.now().timestamp()}"
  }
}
```

3.4.5 Intégration avec l'agent

L'agent utilise ces tools pour répondre à des questions comme :

- « Qu'ai-je aujourd'hui ? » → `get_today_events`

- « Combien de réunions cette semaine ? » → `count_week_events`
 - « Crée une réunion avec Karim demain à 15h » → `create_calendar_event`
 - « Où en est mon action ? » → `check_action_status`
-

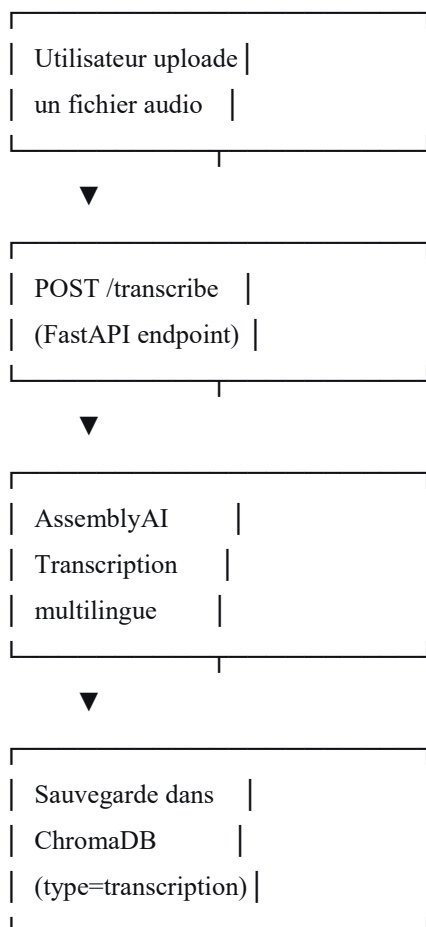
3.5 Fonctionnalité de transcription multi-langues

3.5.1 Objectifs

Répondre au besoin du cahier des charges : « À partir d'un enregistrement audio, de notes ou d'un compte rendu brut, l'agent doit identifier automatiquement les participants, les points abordés, les décisions prises, les actions à mener. »

3.5.2 Chaîne de traitement

text

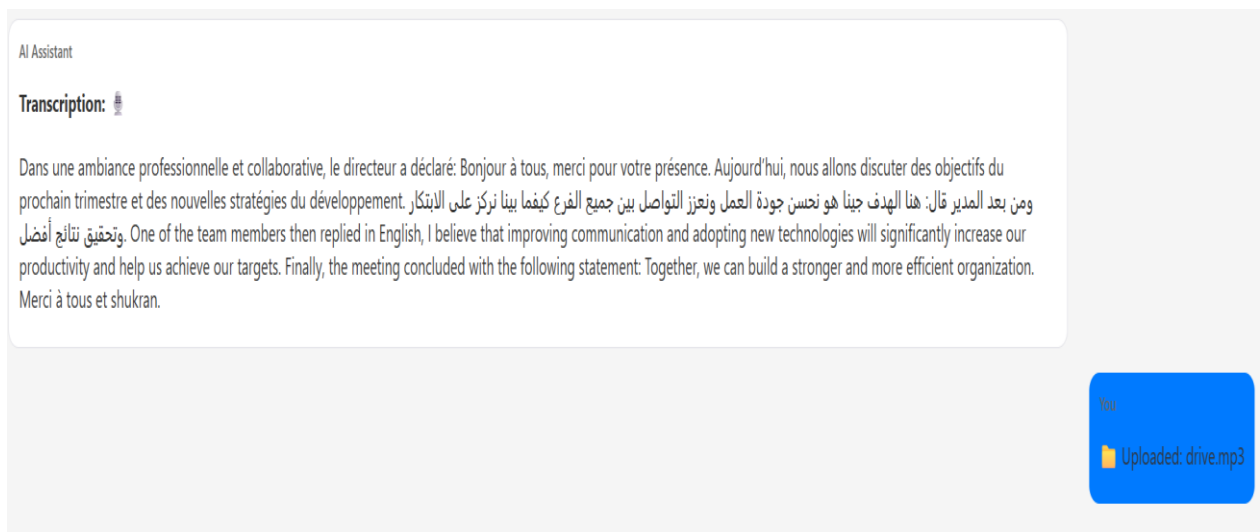


3.5.3 Formats supportés

- .mp3, .m4a, .wav, .flac, .ogg, .aac, .webm

3.5.4 Support multilingue

AssemblyAI gère nativement la détection automatique de la langue. Dans notre test, un fichier `drive.mp3` contenant un mélange de **français, arabe (darija) et anglais** a été transcrit avec succès.



(Figure 3 — Capture d'écran de la transcription multilingue : français, arabe et anglais.)

3.5.5 Indexation dans ChromaDB

Chaque transcription est :

- identifiée par un ID unique (hash SHA-256),
- enrichie de métadonnées (`type=transcription`, `filename`, `date`, `source=AssemblyAI`),
- stockée dans la collection commune (`email_summaries`),
- immédiatement **recherchable** via `search_knowledge`.

Cela permet à l'agent de répondre à des questions comme : « Qu'a-t-on dit sur la stratégie commerciale dans la réunion du 15/09 ? » en s'appuyant sur la transcription indexée.

3.5.6 Enchaînement transcription → analyse → PV

Une fois la transcription disponible, l'agent peut enchaîner :

1. `analyze_meeting_transcript(transcript, meeting_type, participants_emails)`
2. `generate_meeting_minutes(...)` avec les décisions et actions extraites.

Cela réalise le **workflow complet du cahier des charges** : audio → transcription → analyse → PV → diffusion.

Conclusion

Le projet **Agent Intelligent d'Assistance aux Réunions, au Pilotage de la Performance et à la Gestion des Emails** a permis de concevoir et de développer une solution modulaire, proactive et évolutive, répondant à l'ensemble du périmètre fonctionnel du cahier des charges.

Ce projet démontre la **faisabilité technique et l'intérêt métier** d'un agent intelligent unifié pour l'assistance au dirigeant. En combinant **LLM, RAG, tools métiers et validation humaine**, il offre un équilibre rare entre **automatisation** et **contrôle**, entre **proactivité** et **sécurité**. Il constitue une base solide et évolutive vers un véritable **copilote stratégique** pour l'entreprise.

Apports du projet

1. **Unification** : une seule interface pour emails, Drive, Calendar, réunions, SAP.
2. **Mémoire organisationnelle** : ChromaDB conserve et indexe tout l'historique.
3. **Proactivité** : l'agent alerte, relance, challenge.
4. **Traçabilité** : chaque action est journalisée et chaque email sensible est validé.
5. **Multilinguisme** : transcription FR/AR/EN native.
6. **Modularité** : chaque composant (SAP, emails, réunions) est remplaçable indépendamment.

Limites et perspectives

- **SAP réel** : la couche mock doit être remplacée par un connecteur **SAP Service Layer (OData)**.
- **Classification automatique des emails** : à enrichir (règles + LLM).
- **Suivi des dossiers** : manque de données réelles pour tester l'efficacité de l'agent.

Bibliographie / Webographie

1. LangChain Documentation — <https://python.langchain.com>
2. LangGraph Documentation — <https://langchain-ai.github.io/langgraph/>
3. DeepSeek API — <https://api-docs.deepseek.com>
4. ChromaDB Documentation — <https://docs.trychroma.com>
5. AssemblyAI API — <https://www.assemblyai.com/docs>
6. Google Gmail API — <https://developers.google.com/gmail/api>
7. Google Drive API — <https://developers.google.com/drive/api>
8. Google Calendar API — <https://developers.google.com/calendar/api>
9. SAP Business One Service Layer — <https://help.sap.com>
10. FastAPI Documentation — <https://fastapi.tiangolo.com>
11. React Native Documentation — <https://reactnative.dev>
12. Expo Documentation — <https://docs.expo.dev>
13. Sentence-Transformers — <https://www.sbert.net>
14. Pydantic Documentation — <https://docs.pydantic.dev>